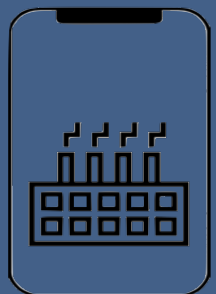
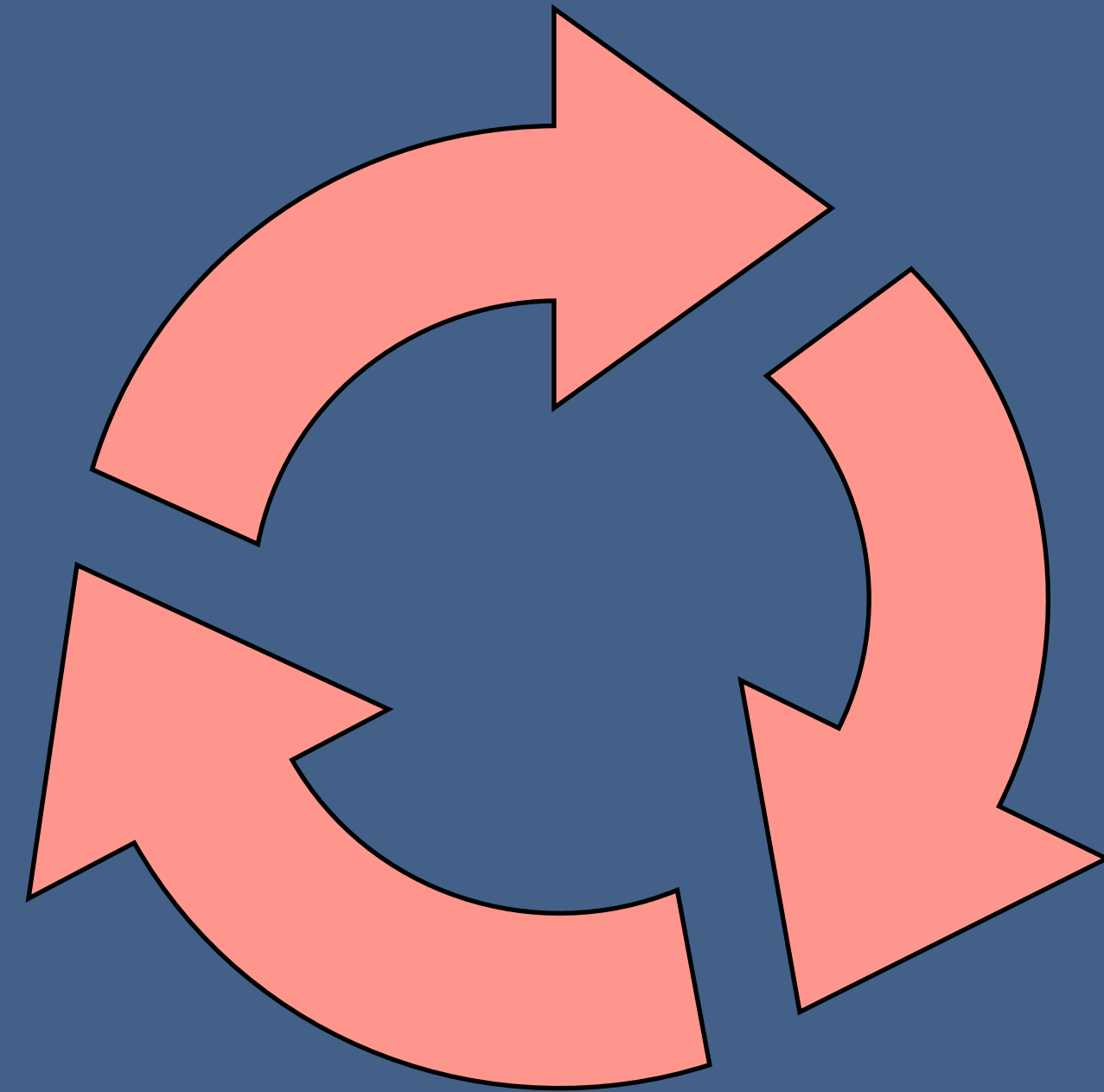


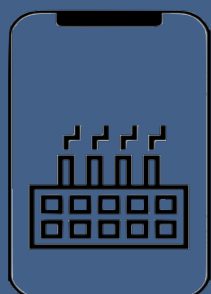
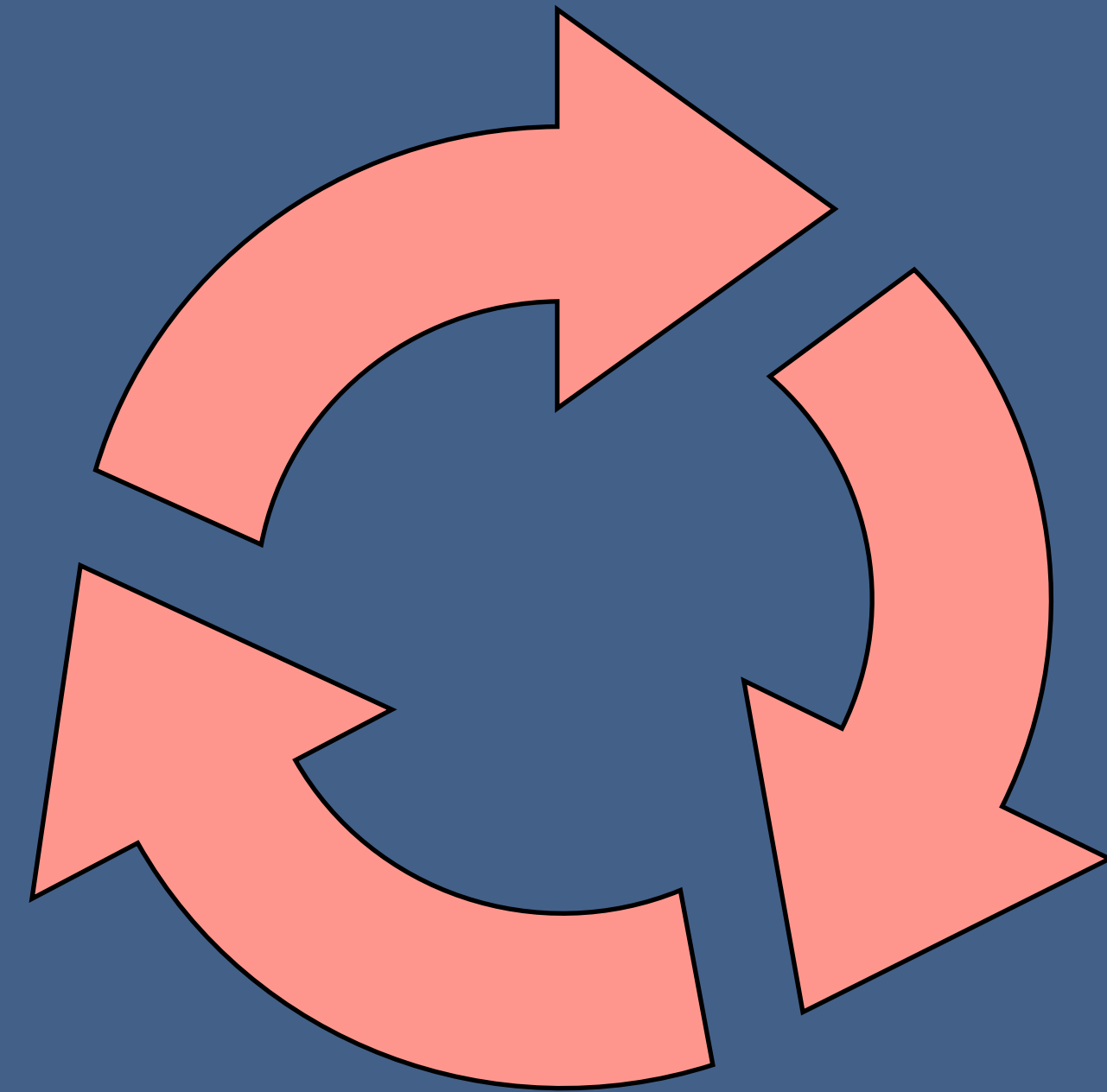
Recursion



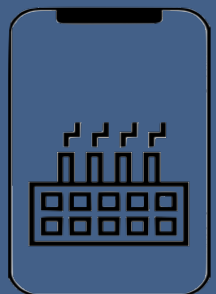
Recursion

Objectives:

- What is Recursion?
- Why do we need Recursion?
- The Logic behind the Recursion
- Recursive vs Iterative Solutions
- How to Write Recursion in 3 Steps?
- Find Fibonacci Series using Recursion

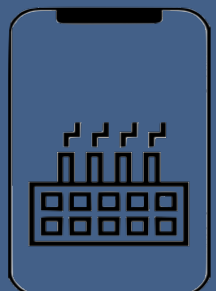
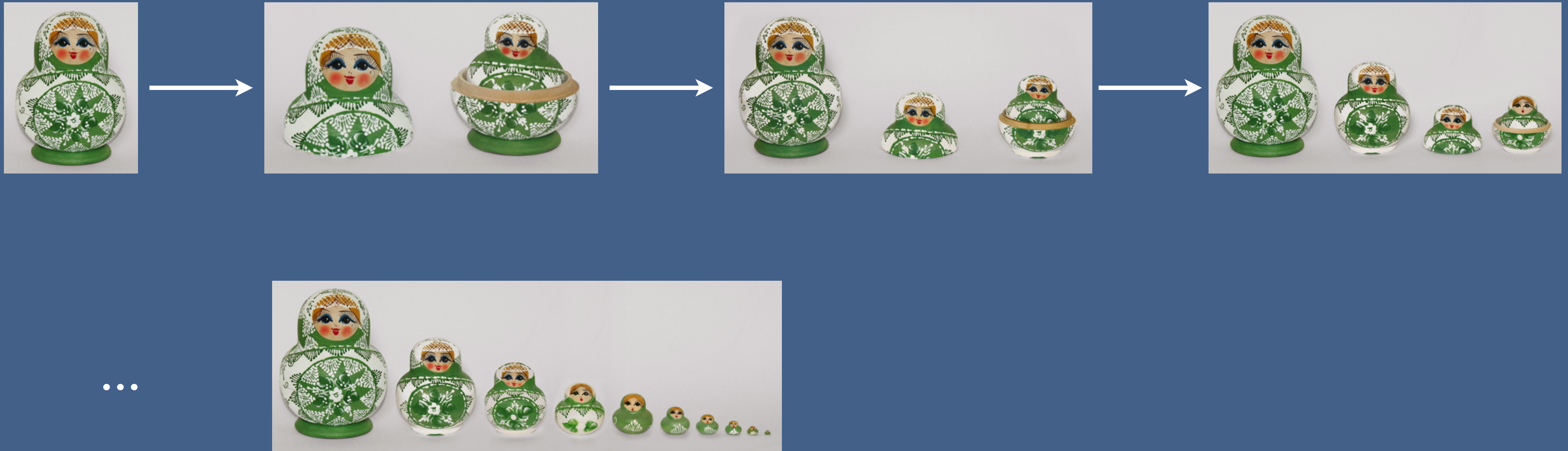


What is Recursion?



What is Recursion?

Recursion : a way of solving a problem by having a function calling itself

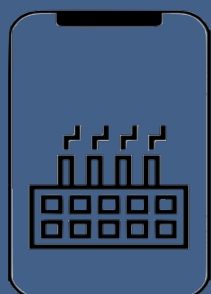


What is Recursion?

Recursion : a way of solving a problem by having a function calling itself



- Performing the same operation multiple times with different inputs
- In every step we try smaller inputs to make the problem smaller.
- Base condition is needed to stop the recursion, otherwise infinite loop will occur.

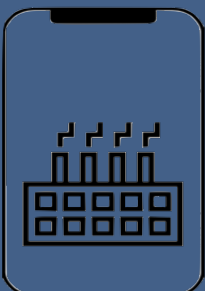


What is Recursion?

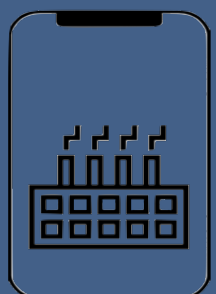
Recursion : a way of solving a problem by having a function calling itself



```
static void openRussianDoll(int doll) {  
    if (doll == 1) {  
        System.out.println("All dolls are opened");  
    } else {  
        openRussianDoll(doll-1);  
    }  
}
```

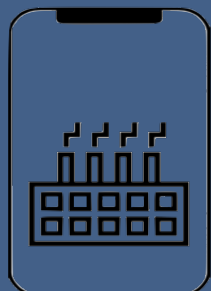


Why we need Recursion?

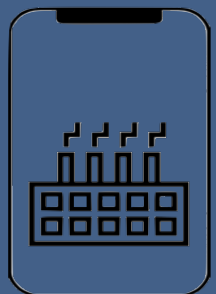


Why we need Recursion?

1. Recursive thinking is really important in programming and it helps you break down big problems into smaller ones and easier to use
 - when to choose recursion?
 - If you can divide the problem into similar sub problems
 - Design an algorithm to compute nth...
 - Write code to list the n...
 - Implement a method to compute all.
 - Practice
2. The prominent usage of recursion in data structures like trees and graphs.
3. Interviews
4. It is used in many algorithms (divide and conquer, greedy and dynamic programming)



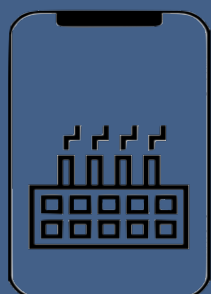
The Logic Behind Recursion



The Logic Behind Recursion

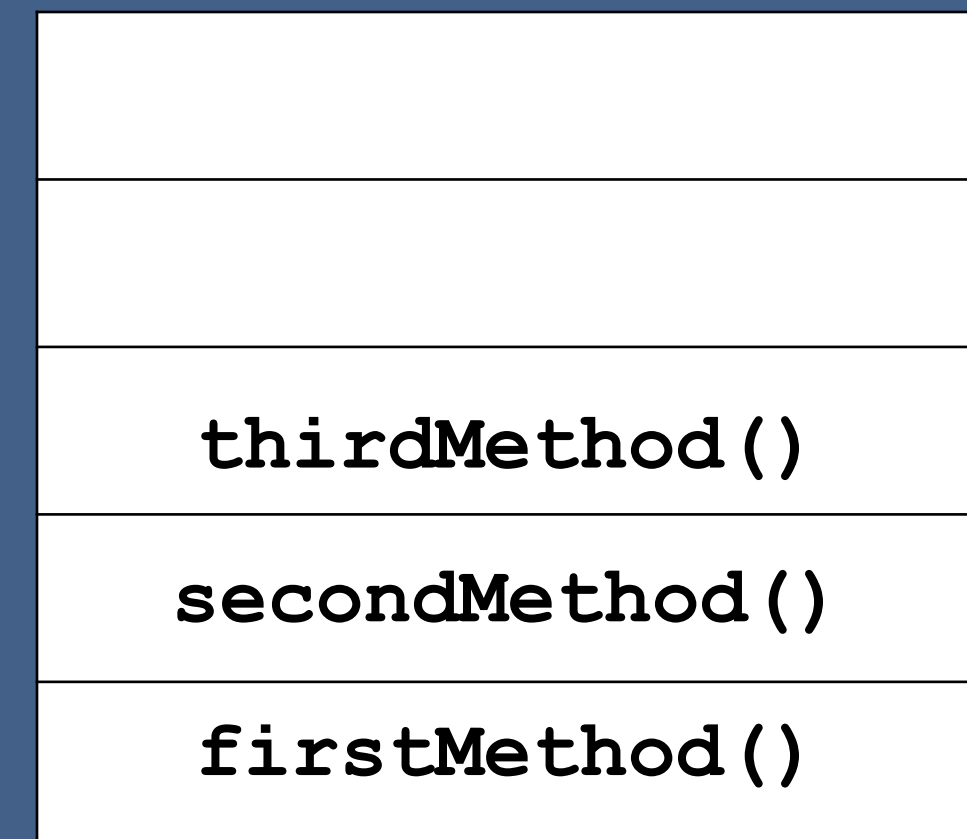
1. A method calls it self
2. Exit from infinite loop

```
static string recursionMethod(String[] parameters) {  
    if (exit from condition satisfied) {  
        return some value;  
    } else {  
        recursionMethod(modified parameters);  
    }  
}
```

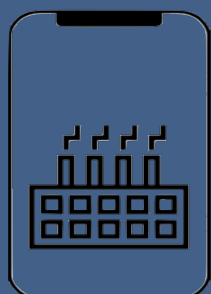


The Logic Behind Recursion

```
static void firstMethod() {  
    secondMethod();  
    System.out.println("I am the first Method");  
}  
  
static void secondMethod() {  
    thirdMethod();  
    System.out.println("I am the second Method");  
}  
  
static void thirdMethod() {  
    fourthMethod();  
    System.out.println("I am the third Method");  
}  
  
static void fourthMethod() {  
    System.out.println("I am the fourth Method");  
}
```



STACK Memory

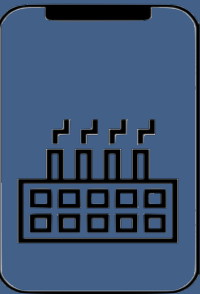
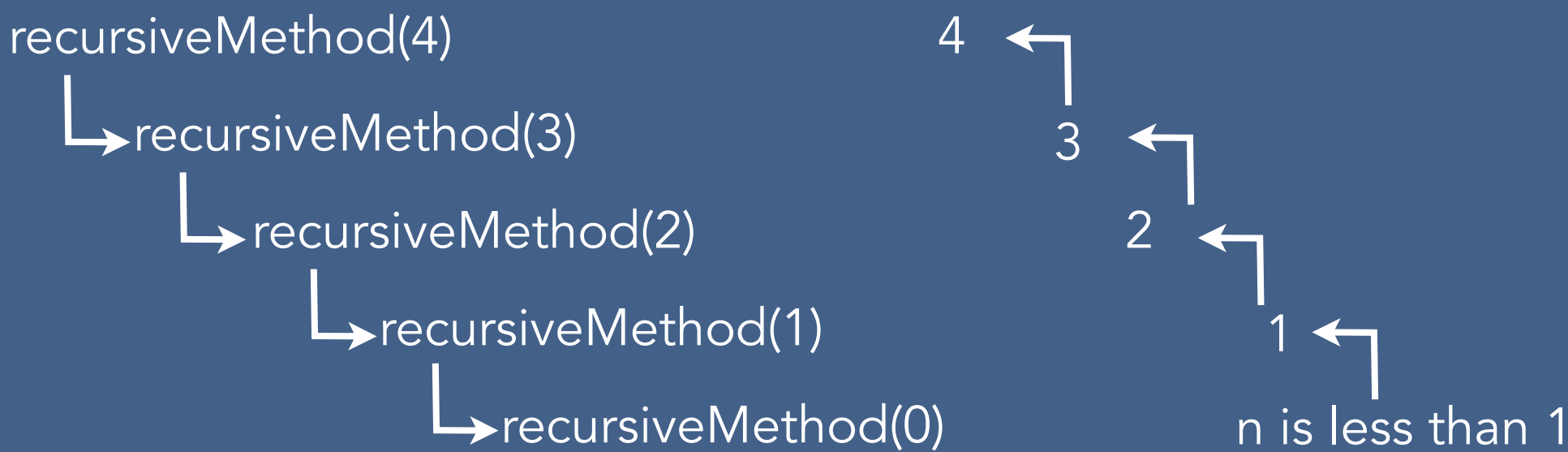


The Logic Behind Recursion

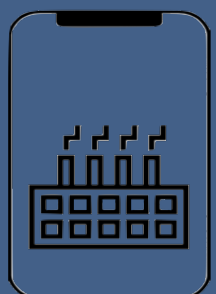
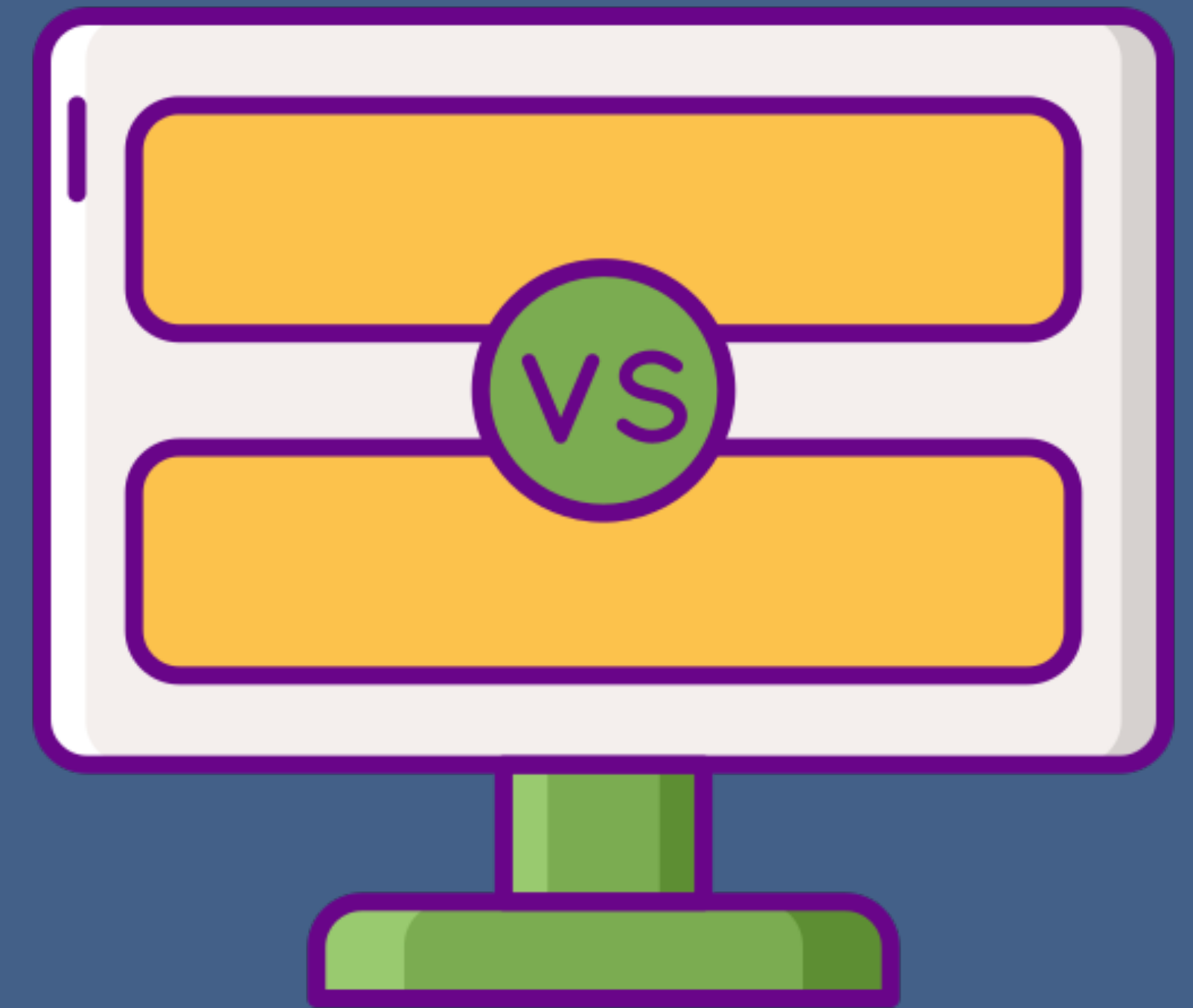
```
static void recursiveMethod(int n) {  
    if (n<1) {  
        System.out.println("n is less than 1");  
    } else {  
        recursiveMethod(n-1);  
        System.out.println(n);  
    }  
}
```

recursiveMethod(1)
recursiveMethod(2)
recursiveMethod(3)
recursiveMethod(4)

STACK Memory



Recursive vs Iterative Solutions



Recursive vs Iterative Solutions

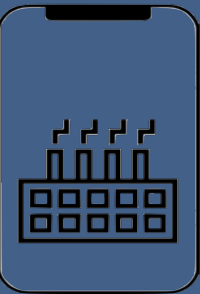
Recursive

```
static int powerOfTwo(int n) {
    if (n==0) {
        return 1;
    } else {
        var power = 2*powerOfTwo(n-1);
        return power;
    }
}
```

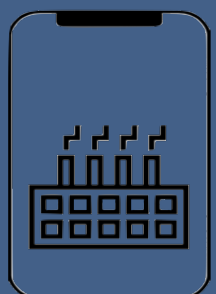
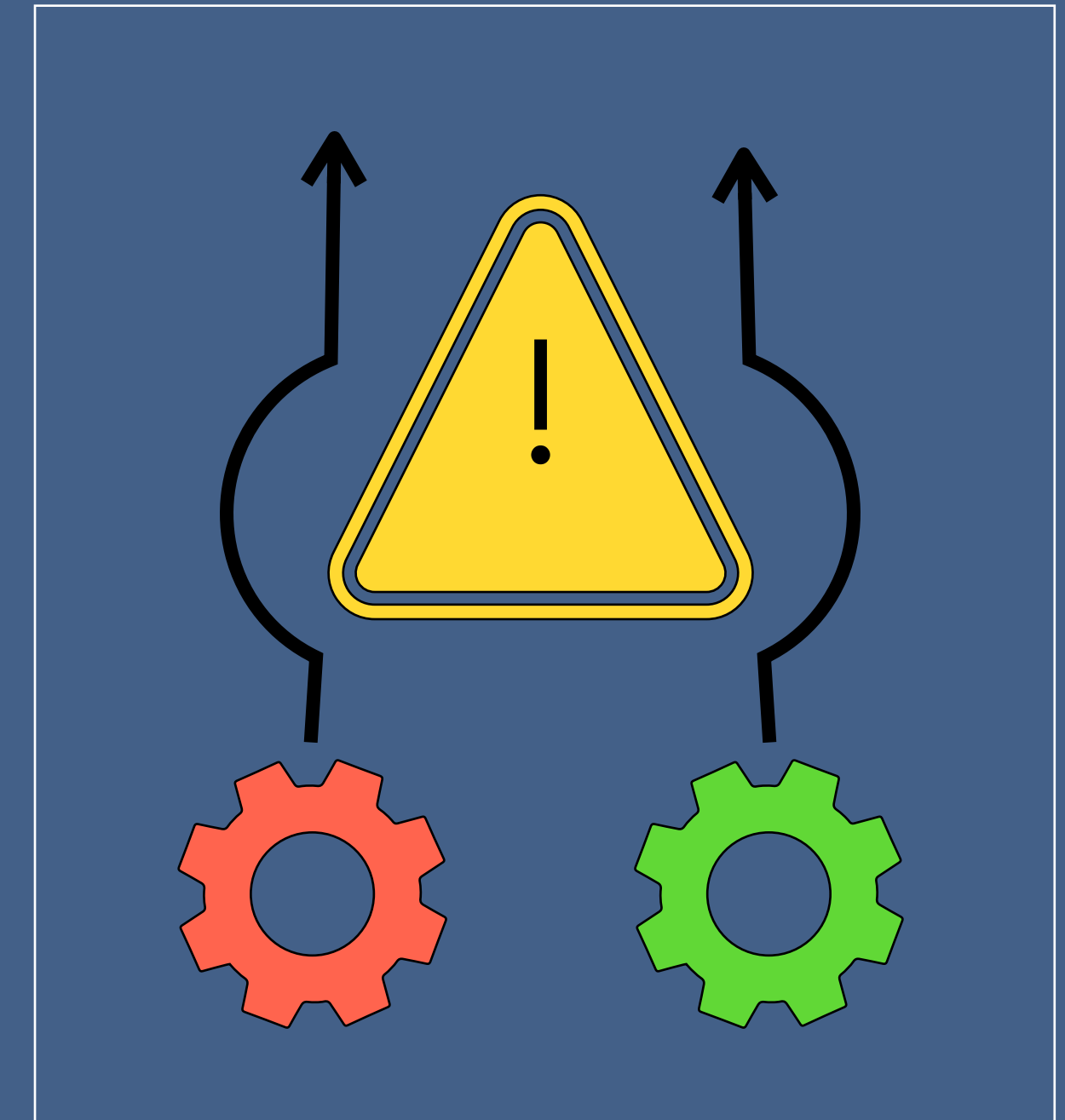
Iterative

```
static int powerOfTwoIT(int n) {
    var i = 0;
    var power = 1;
    while (i < n) {
        power = power * 2;
        i = i + 1;
    }
    return power;
}
```

Points	Recursion	Iteration	
Space efficient?	No	Yes	No stack memory require in case of iteration
Time efficient?	No	Yes	In case of recursion system needs more time for pop and push elements to stack memory which makes recursion less time efficient
Easy to code?	Yes	No	We use recursion especially in the cases we know that a problem can be divided into similar sub problems.



When to Use/Avoid Recursion?

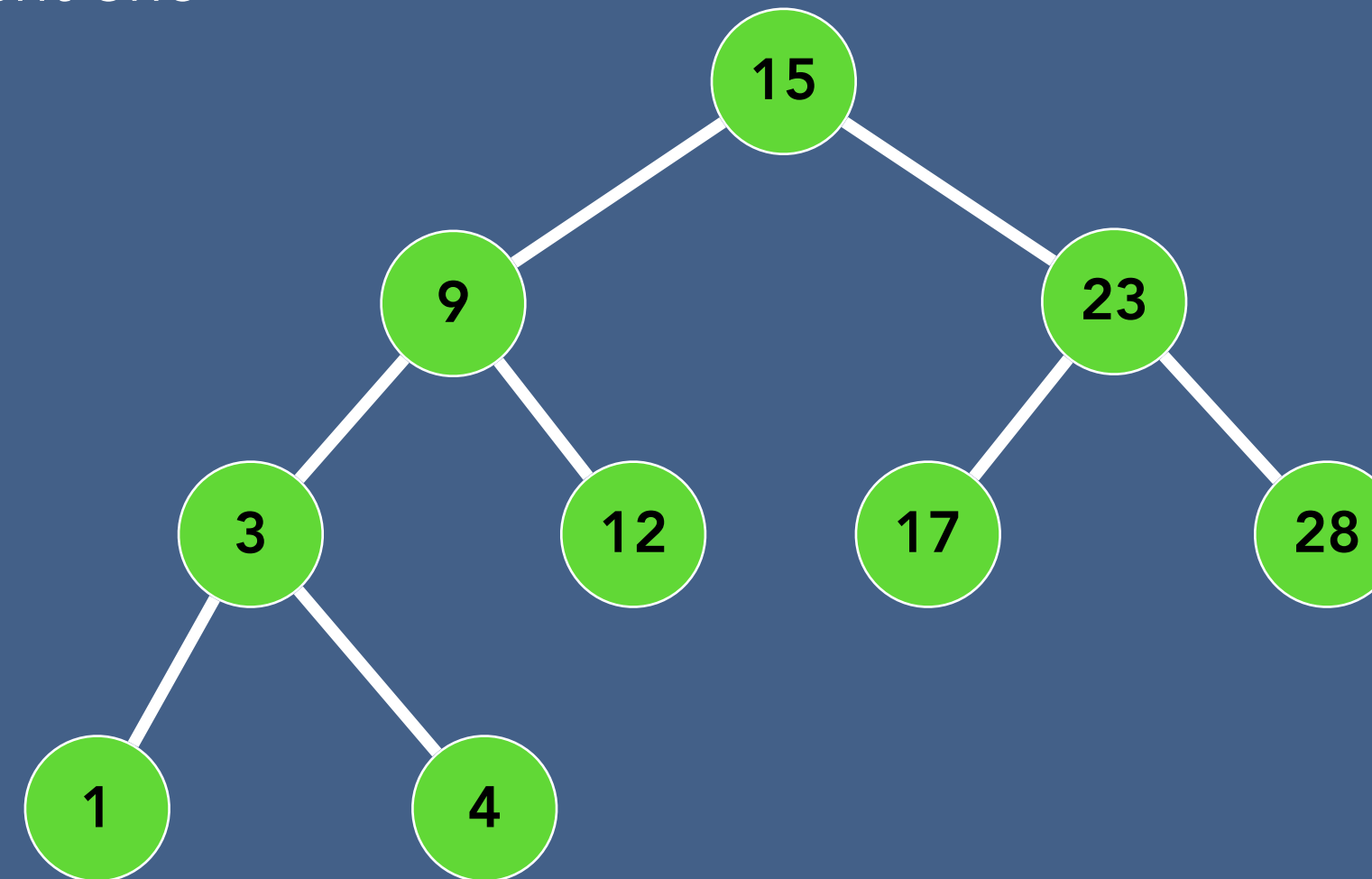


When to Use/Avoid Recursion?

When to use it?

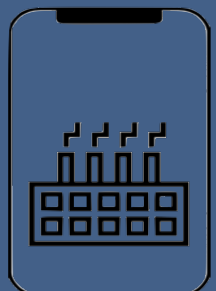
- When we can easily breakdown a problem into similar subproblem
- When we are fine with extra overhead (both time and space) that comes with it
- When we need a quick working solution instead of efficient one
- When traverse a tree
- When we use memoization in recursion

- preorder tree traversal : 15, 9, 3, 1, 4, 12, 23, 17, 28

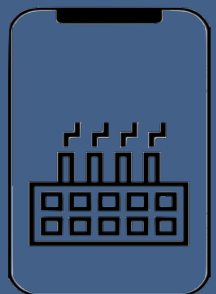
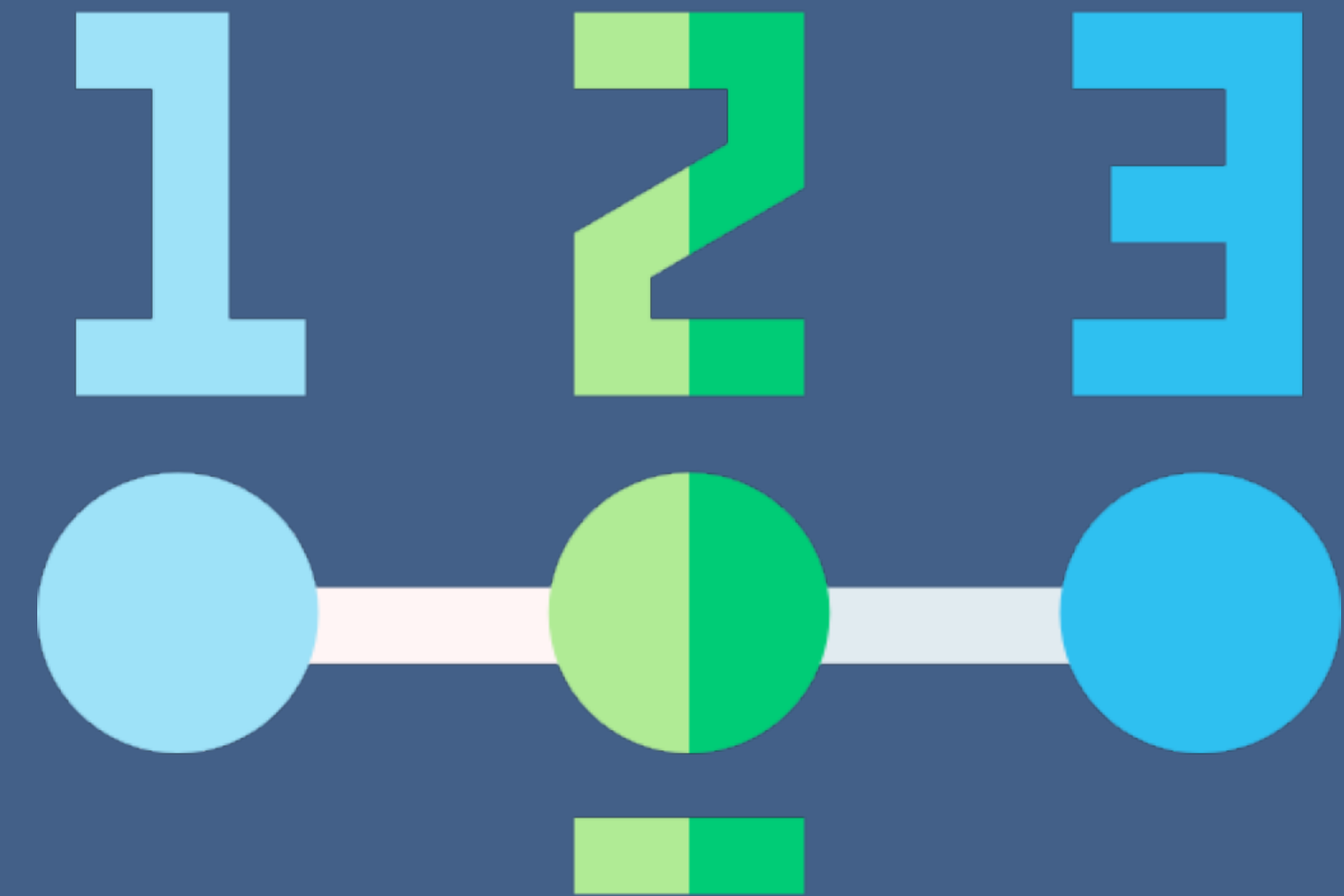


When avoid it?

- If time and space complexity matters for us.
- Recursion uses more memory. If we use embedded memory. For example an application that takes more memory in the phone is not efficient
- Recursion can be slow



Write Recursion in 3 Steps



Write Recursion in 3 Steps

Factorial

- It is the product of all positive integers less than or equal to n.
- Denoted by n! (Christian Kramp in 1808).
- Only positive numbers.
- $0! = 1$.



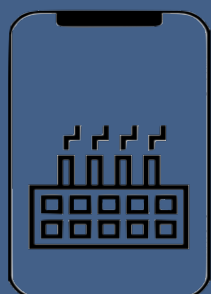
Example 1

$$4! = 4 * 3 * 2 * 1 = 24$$

Example 2

$$10! = 10 * 9 * 8 * 7 * 6 * 5 * 4 * 3 * 2 * 1 = 36,28,800$$

$$n! = n * (n-1) * (n-2) * \dots * 2 * 1$$



Write Recursion in 3 Steps

Step 1 : Recursive case - the flow

$$n! = n * (n-1) * (n-2) * \dots * 2 * 1 \longrightarrow \boxed{n! = n * (n-1)!}$$

$\downarrow$
 $(n-1)!$

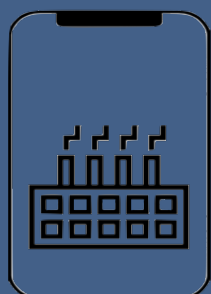
$$(n-1)! = (n-1) * (n-1-1) * (n-1-2) * \dots * 2 * 1 = (n-1) * (n-2) * (n-3) * \dots * 2 * 1$$

Step 2 : Base case - the stopping criterion

- $0! = 1$
- $1! = 1$

Step 3 : Unintentional case - the constraint

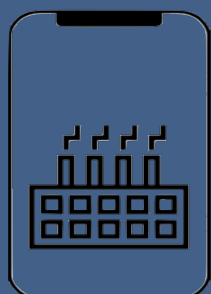
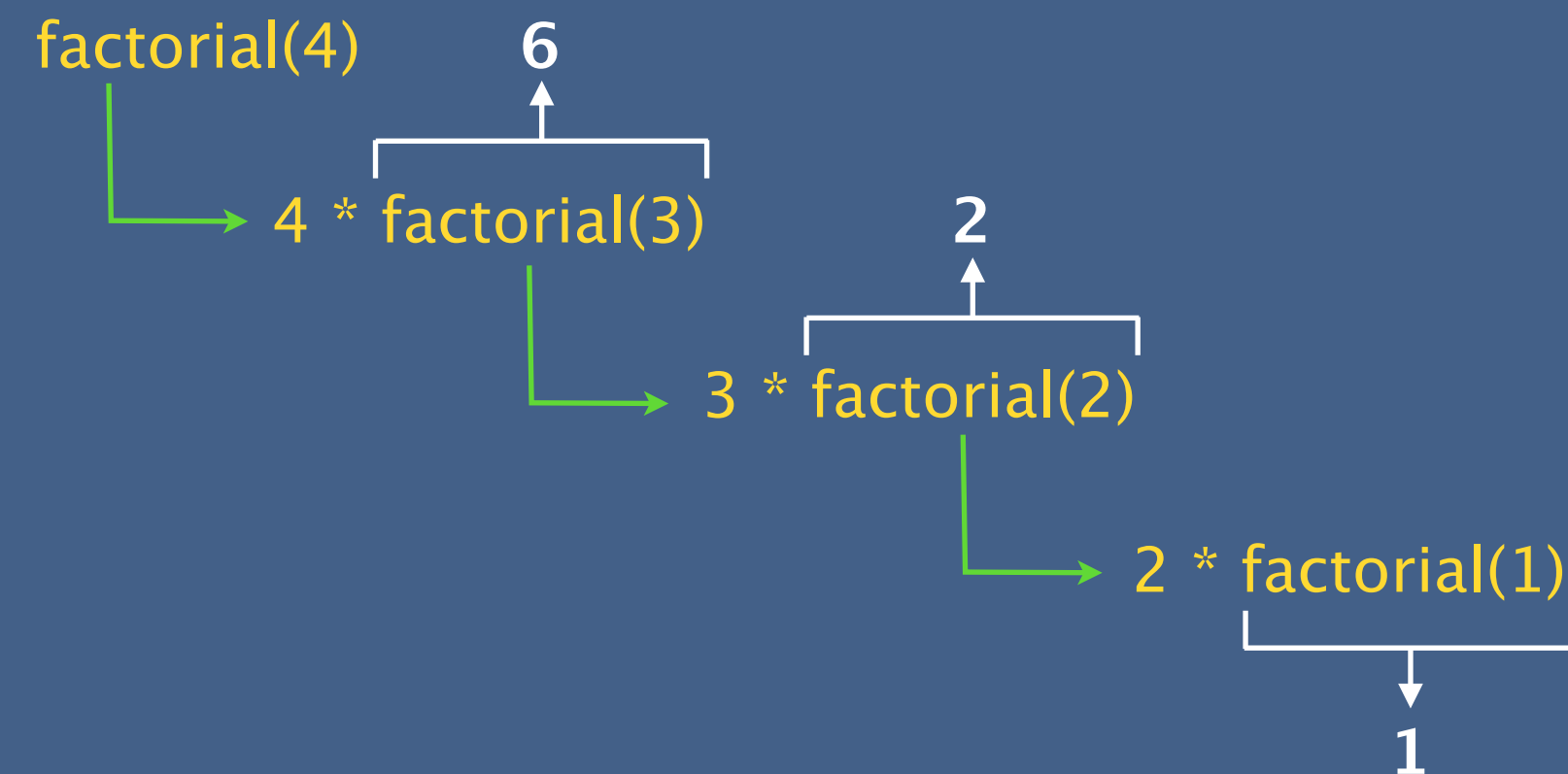
- `factorial(-1)` ??
- `factorial(-2)` ??



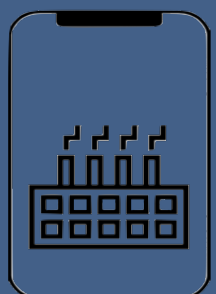
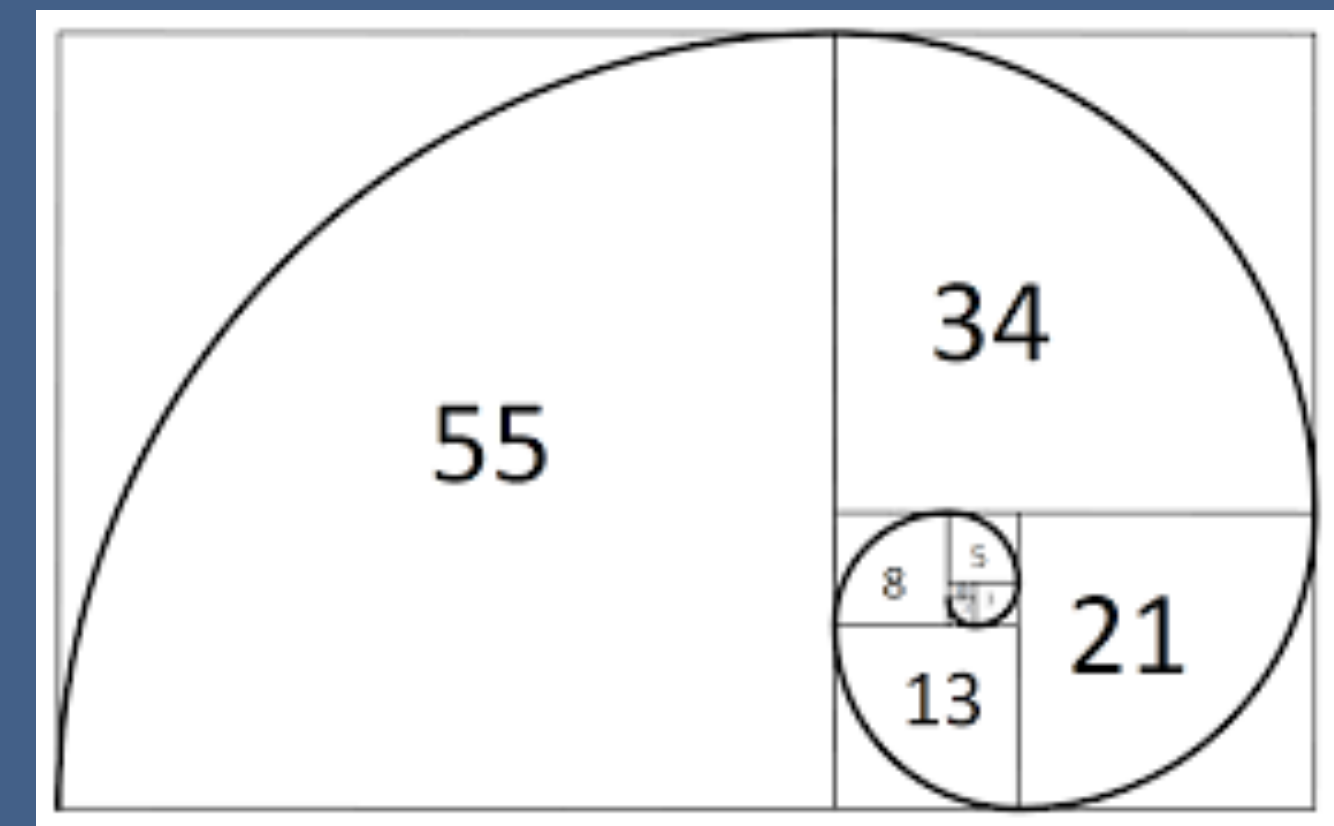
Write Recursion in 3 Steps

```
public int factorial(int n) {  
    if (n < 0) {  
        return -1;  
    }  
    if (n == 0 || n == 1) {  
        return 1;  
    }  
    return n * factorial(n-1);  
}
```

factorial(4) = 24



Fibonacci using Recursion



Write Fibonacci Recursion in 3 Steps

Fibonacci numbers

- Fibonacci sequence is a sequence of numbers in which each number is the sum of the two preceding ones and the sequence starts from 0 and 1

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89...

Step 1 : Recursive case - the flow

$$5 = 3 + 2$$

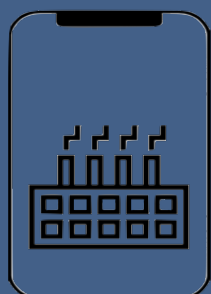
$$f(n) = f(n-1) + f(n-2)$$

Step 2 : Base case - the stopping criterion

- 0 and 1

Step 3 : Unintentional case - the constraint

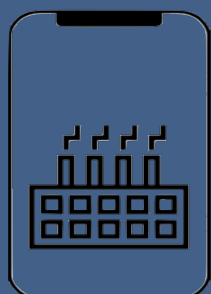
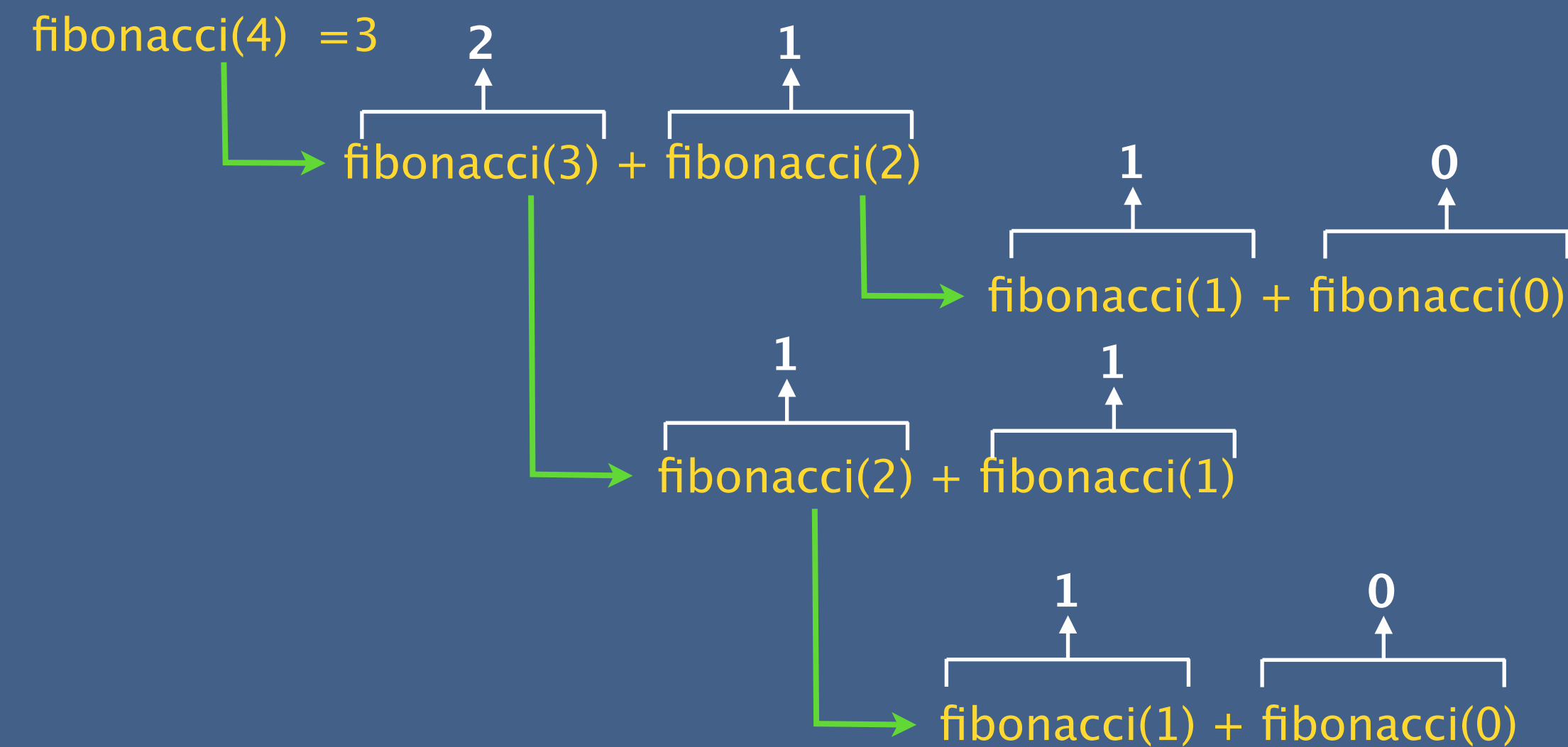
- fibonacci(-1) ??
- fibonacci(1.5) ??



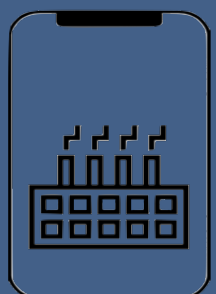
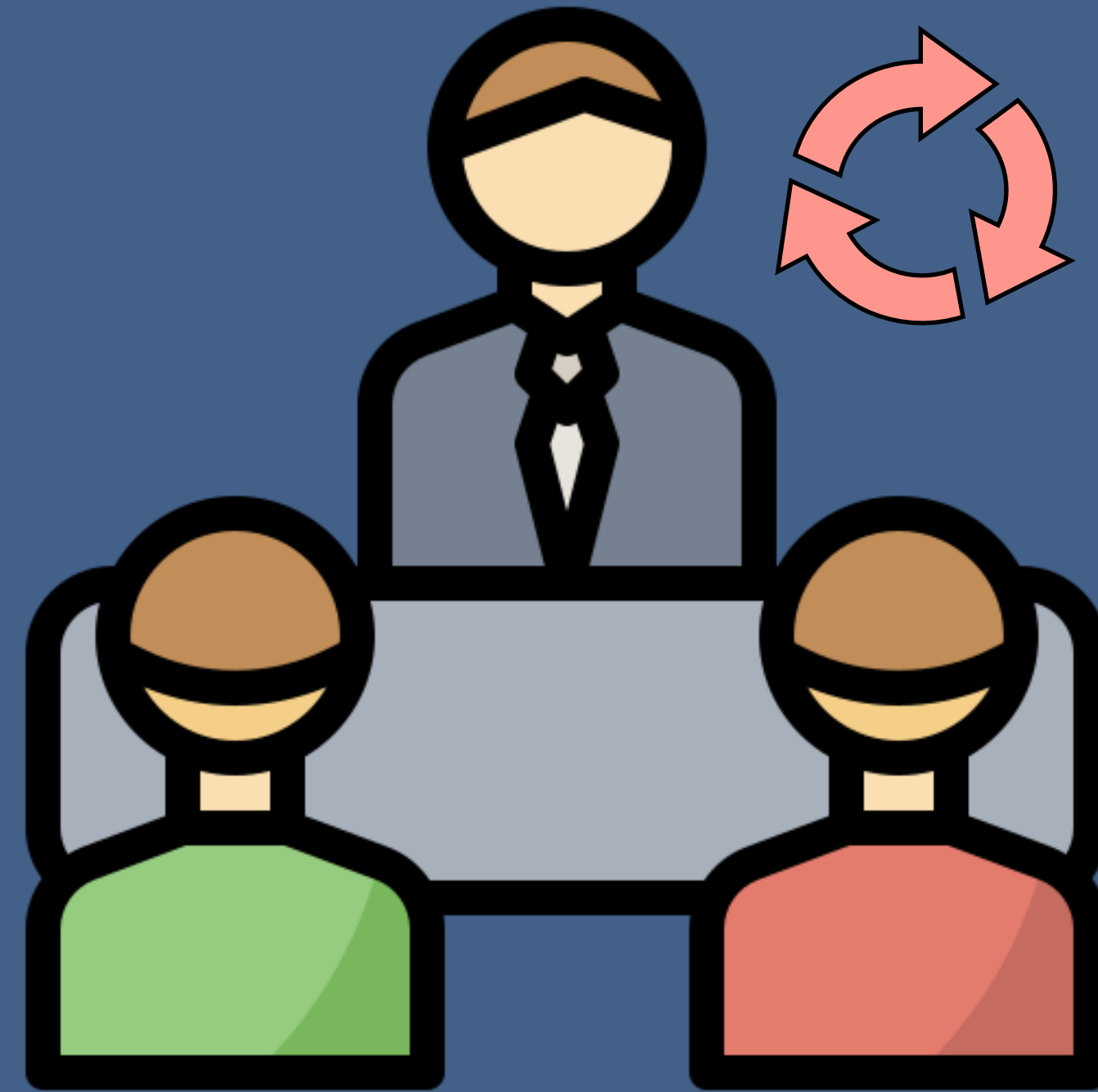
Write Fibonacci Recursion in 3 Steps

```
public int fibonacci(int n) {  
    if (n < 0) {  
        return -1;  
    }  
    if (n == 0 || n == 1) {  
        return n;  
    }  
    return fibonacci(n-1) + fibonacci(n-2);  
}
```

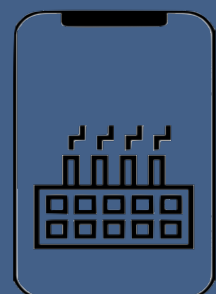
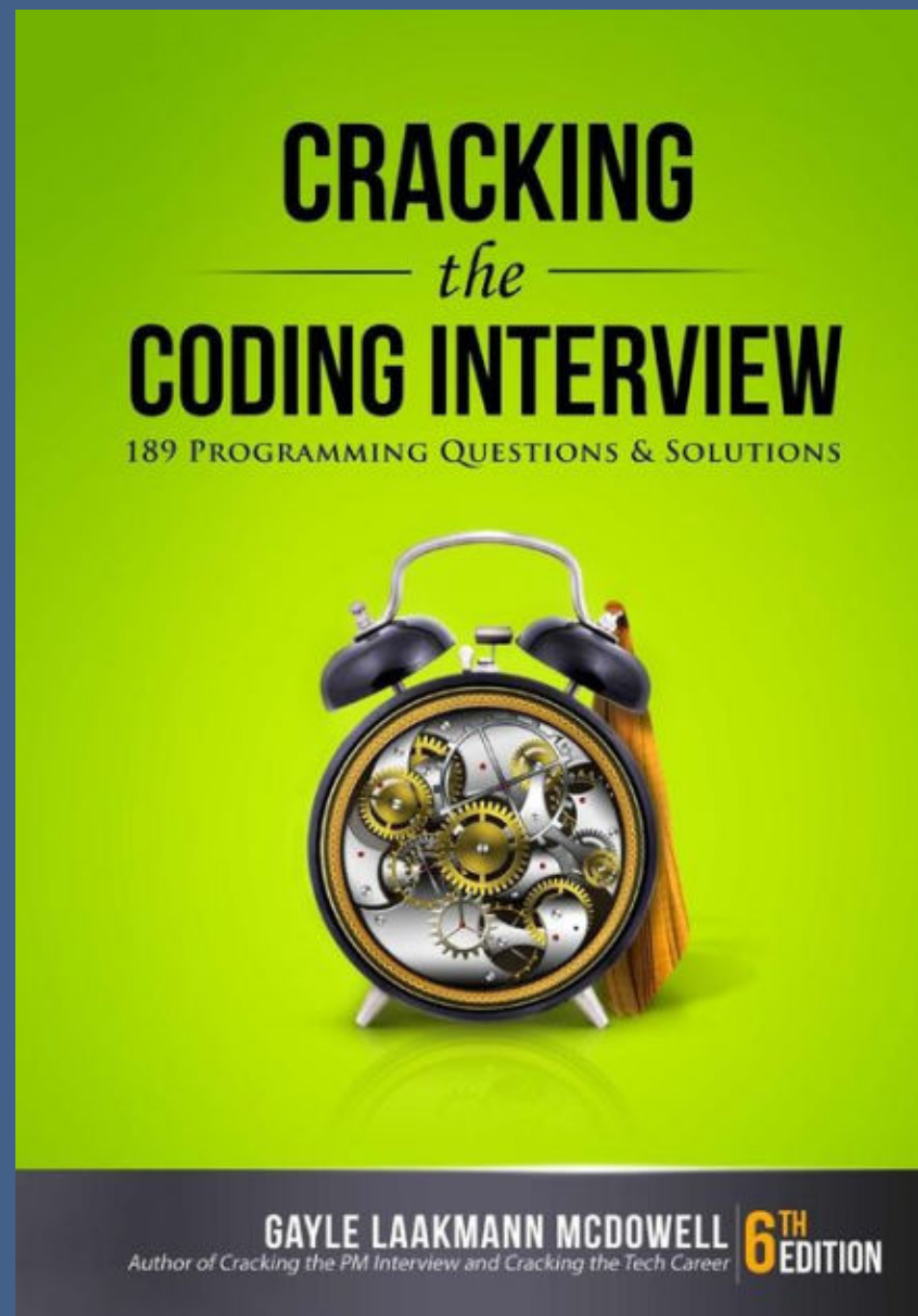
0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89...



Recursion Interview Questions



Recursion Interview Questions



Recursion Interview Questions - 1

How to find the sum of digits of a positive integer number using recursion ?

Step 1 : Recursive case - the flow

10 $10/10 = 1$ and Remainder = 0

54 $54/10 = 5$ and Remainder = 4

112 $112/10 = 11$ and Remainder = 2

$11/10 = 1$ and Remainder = 1

$$f(n) = n \% 10 + f(n/10)$$

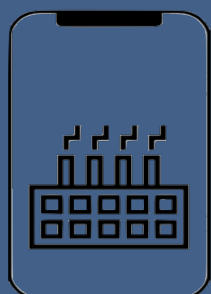
Step 2 : Base case - the stopping criterion

– $n = 0$

Step 3 : Unintentional case - the constraint

– `sumofDigits(-11)` ??

– `sumofDigits(1.5)` ??



Recursion Interview Questions - 2

How to calculate power of a number using recursion?

Step 1 : Recursive case - the flow

$$x^n = x * x * x * \dots (n \text{ times}) * x$$

$$2^4 = 2 * 2 * 2 * 2$$

$$x^n = x * x^{n-1}$$

$$x^a * x^b = x^{a+b}$$

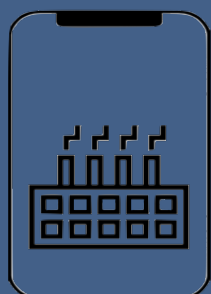
$$x^3 * x^4 = x^{3+4}$$

Step 2 : Base case - the stopping criterion

- $n = 0$
- $n = 1$

Step 3 : Unintentional case - the constraint

- $\text{power}(2, 1.2)$??
- $\text{power}(2, -1)$??



Recursion Interview Questions - 3

How to find GCD (Greatest Common Divisor) of two numbers using recursion?

Step 1 : Recursive case - the flow

GCD is the largest positive integer that divides the numbers without a remainder

$$\text{gcd}(8,12) = 4$$

$$8 = 2 * 2 * 2$$

$$12 = 2 * 2 * 3$$

Euclidean algorithm

$$\text{gcd}(48,18)$$

$$\text{Step 1 : } 48/18 = 2 \text{ remainder } 12$$

$$\text{Step 2 : } 18/12 = 1 \text{ remainder } 6$$

$$\text{Step 3 : } 12/6 = 2 \text{ remainder } 0$$

$$\text{gcd}(a, 0)=a$$

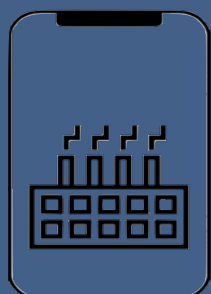
$$\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$$

Step 2 : Base case - the stopping criterion

$$- b = 0$$

Step 3 : Unintentional case - the constraint

– Positive integers



Recursion Interview Questions - 4

How to convert a number from Decimal to Binary using recursion

Step 1 : Recursive case - the flow

- Step 1 : Divide the number by 2
- Step 2 : Get the integer quotient for the next iteration
- Step 3 : Get the remainder for the binary digit
- Step 3 : Repeat the steps until the quotient is equal to 0

10 to binary 1000

Division by 2	Quotient	Remainder
10/2	5	0
5/2	2	1
2/2	1	0
1/2	0	1

$f(n) = n \bmod 2 + 10 * f(n/2)$

$101 * 10 + 0 = 1010$
 $10 * 10 + 1 = 101$
 $1 * 10 + 0 = 10$

